# Instituto Tecnológico Sudamericano

## Carrera

Desarrollo de Software

## Actividad:

Tarea en Clase , Promt usados

## Nombre:

Bryam Paul Villalva Flores

## Materia:

Herramientas Informáticas Para El Despliegue De Diagramas

## Fecha:

10 de Octubre del 2026

# Introducción

Este documento presenta un registro del uso de herramientas de inteligencia artificial como apoyo durante el desarrollo de una API básica para administrar usuarios mediante NestJS. Se describen las consultas realizadas, la finalidad de cada una y los resultados obtenidos durante el proceso de configuración, organización y prueba del recurso.

# 1. Objetivo del trabajo

Documentar el proceso seguido para construir un CRUD de usuarios en NestJS, identificando cómo los prompts ayudaron a resolver dudas sobre los comandos de generación, la organización de archivos, el manejo temporal de datos, la validación de información y las pruebas de los endpoints con Postman.

# 2. Registro de consultas y resultados

## Consulta 1. Generación del recurso de usuarios

> ***Prompt:*** *Prompt de referencia: "¿Cómo puedo crear correctamente el recurso users en NestJS y generar sus archivos principales?"*

**Finalidad:** Propósito: Obtener una estructura inicial para trabajar con un módulo, controlador y servicio que permitan administrar usuarios.

**Palabras clave:** Términos clave: NestJS, CLI, generate resource, controller, service, CRUD.

**Resultado:** Resultado: Se identificó que la creación del proyecto y la generación de un recurso son comandos diferentes. La orientación permitió organizar la estructura inicial del recurso y reconocer los archivos que intervienen en las operaciones CRUD.

## Consulta 2. Organización de rutas en un archivo auxiliar

> ***Prompt:*** *Prompt de referencia: "Quiero guardar en un archivo llamado temporal.ts un arreglo con las rutas de mi recurso de usuarios. ¿Cómo puedo organizarlo?"*

**Finalidad:** Propósito: Mantener una lista de endpoints en un archivo independiente para consultarlos durante la documentación o las pruebas.

**Palabras clave:** Términos clave: temporal.ts, endpoints, rutas HTTP, export const, GET, POST, PATCH, DELETE.

**Resultado:** Resultado: Se planteó un arreglo exportable con las rutas principales del recurso. También se distinguió entre registrar una ruta como texto y crear una ruta funcional dentro del controlador.

## Consulta 3. Definición de la estructura User

> ***Prompt:*** *Prompt de referencia: "Necesito definir una clase User con sus propiedades y utilizarla junto con el arreglo de endpoints. ¿Qué estructura debería tener?"*

**Finalidad:** Propósito: Establecer una representación básica del usuario y comprender cómo exportar elementos desde un archivo TypeScript.

**Palabras clave:** Términos clave: class User, propiedades, constructor, TypeScript, export, entidad.

**Resultado:** Resultado: Se revisó la forma de declarar una clase para representar los datos de un usuario. La consulta ayudó a diferenciar la definición de la estructura de datos de la lógica que crea, consulta o modifica usuarios.

## Consulta 4. Corrección del constructor y almacenamiento

> ***Prompt:*** *Prompt de referencia: "Ya creé la clase User, pero los datos no aparecen en temporal.ts después de realizar una petición. ¿Cómo debo manejar esa información?"*

**Finalidad:** Propósito: Comprender por qué un archivo TypeScript no cambia automáticamente al recibir peticiones y dónde guardar los datos durante la ejecución.

**Palabras clave:** Términos clave: constructor, Object.assign, UserService, arreglo users, memoria, persistencia.

**Resultado:** Resultado: Se aclaró que temporal.ts contiene definiciones escritas en el código y no funciona como una base de datos dinámica. Para una práctica sencilla, los usuarios pueden mantenerse temporalmente en un arreglo dentro del servicio; estos datos no son permanentes y se pierden al reiniciar la aplicación.

### Consulta 5. Validación de datos y pruebas con Postman

> ***Prompt:*** *Prompt de referencia: "Los métodos solo muestran mensajes y no están creando usuarios.*
>
> *¿Cómo puedo implementar la lógica y probarla con Postman?"*

**Finalidad:** Propósito: Pasar de respuestas de ejemplo a operaciones que reciban y procesen datos enviados mediante solicitudes HTTP.

**Palabras clave:** Términos clave: CreateUserDto, UpdateUserDto, class-validator, IsString, IsEmail, JSON, Postman.

**Resultado:** Resultado: Se identificó la necesidad de definir DTOs para estructurar los datos de entrada, conectar el controlador con el servicio y comprobar las operaciones enviando solicitudes HTTP. La validación puede incluir campos como nombre y correo electrónico.

## 3. Componentes principales del desarrollo

• **Recurso de usuarios:** reúne el controlador, el servicio y los archivos necesarios para organizar las operaciones.

• **Endpoints:** representan las rutas HTTP utilizadas para crear, listar, consultar, actualizar y eliminar usuarios.

• **Clase User:** describe de forma básica los datos que tendrá cada usuario.

• **DTOs:** definen la estructura esperada para los datos recibidos y pueden incorporar reglas de validación.

• **UserService:** concentra la lógica de las operaciones. En el ejemplo documentado, un arreglo en memoria permite probar el funcionamiento sin una base de datos.

• **Postman:** permite enviar solicitudes y revisar las respuestas de la API durante las pruebas.

## 4. Conclusiones

• El uso de prompts permitió dividir el desarrollo en tareas concretas y resolver dudas de forma progresiva, desde la creación del recurso hasta la comprobación de las peticiones.

• Se comprendió que los archivos auxiliares con constantes o clases no almacenan automáticamente la información enviada a la API. Para conservar datos durante la ejecución se necesita una estructura de almacenamiento, aunque un arreglo en memoria solo sirve como solución temporal.

• La separación entre controlador, servicio y DTO facilita la organización del código: el controlador recibe las solicitudes, el servicio ejecuta la lógica y los DTOs describen los datos esperados.

• Finalmente, Postman es útil para comprobar los endpoints, pero es necesario verificar los códigos de respuesta y el contenido devuelto para confirmar que cada operación funciona correctamente.

## 5. Recomendaciones

- Escribir prompts específicos, indicando el framework, el archivo involucrado y el error observado.
- Revisar y comprender el código generado antes de integrarlo al proyecto.
- Probar cada endpoint por separado y utilizar datos válidos e inválidos para revisar el comportamiento de la API.
- Si se requiere conservar los usuarios después de reiniciar el servidor, incorporar posteriormente una base de datos en lugar de depender de un arreglo en memoria.